



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

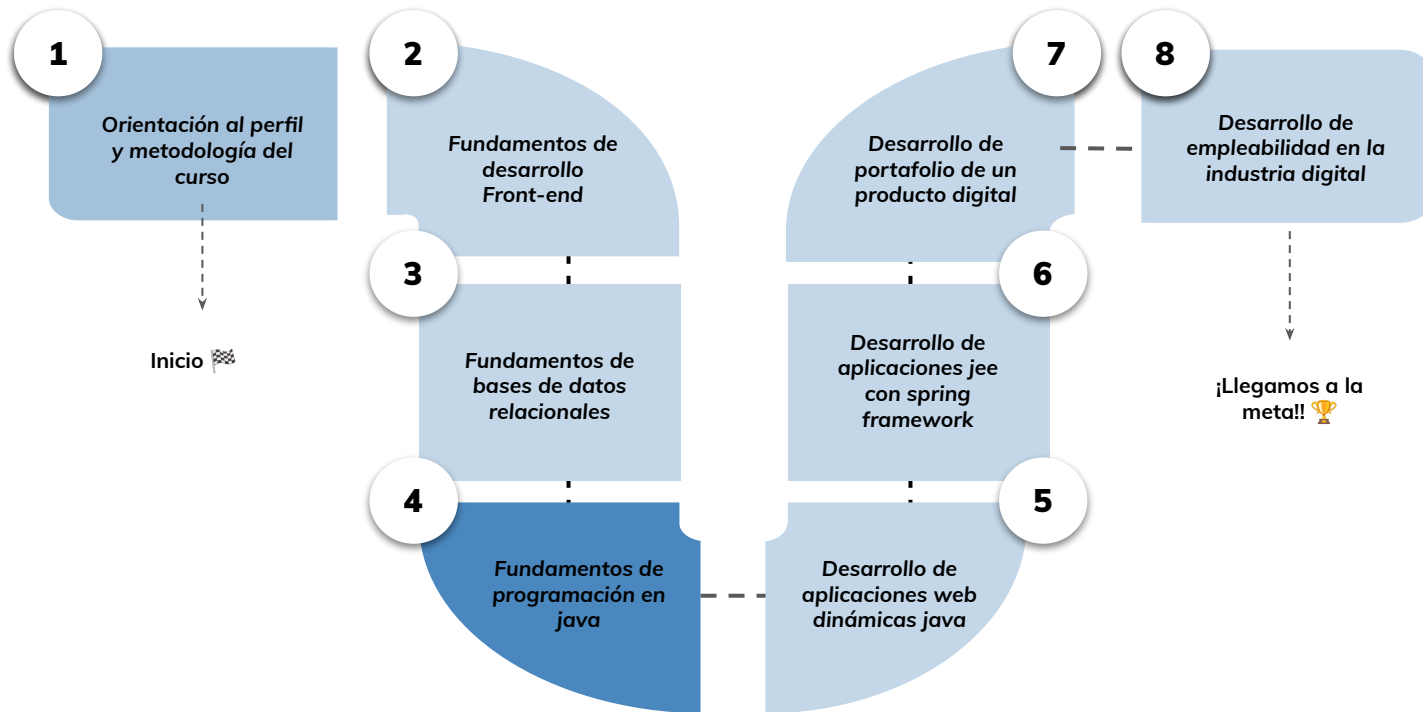


› Algoritmos- Parte 2

Aprendizaje Esperado 2: Estructurar un algoritmo lógico utilizando estructuras de control y expresiones para dar solución a un problema de baja complejidad acorde al lenguaje java.

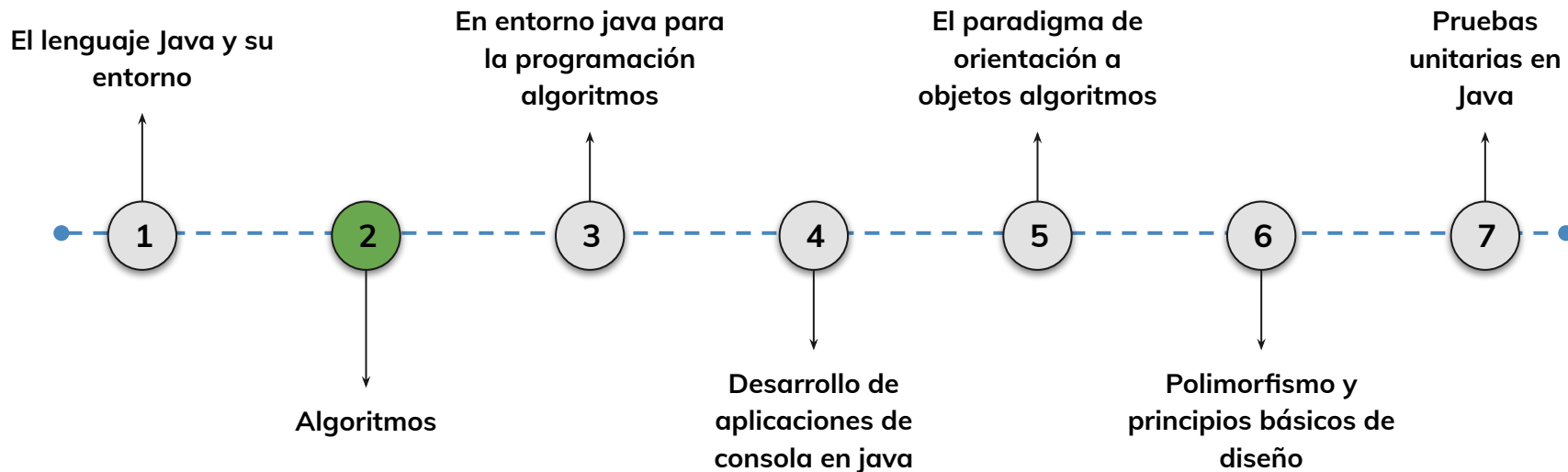
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Estructuración de algoritmos con estructuras repetitivas, subprogramas y diagramas

Exploraremos nuevas formas de estructurar algoritmos mediante bucles (Mientras y Para), la creación de funciones y procedimientos, el uso de arreglos y matrices, y la representación visual de procesos a través de diagramas de flujo. Se trabajará con pseudocódigo para consolidar los conceptos.

Estructuras repetitivas y diagramas de flujo

Bucle Mientras y Para

Diagramas de flujo y su simbología

Subprogramas y estructuras de datos

Funciones y procedimientos

Arreglos unidimensionales y bidimensionales



Objetivos de aprendizaje

¿Qué aprenderás?



- Aplicar estructuras de control repetitivas para resolver tareas iterativas.
- Representar algoritmos mediante diagramas de flujo.
- Crear y utilizar arreglos unidimensionales y bidimensionales.
- Implementar funciones que reciben parámetros y retornan resultados.
- Diferenciar el uso de funciones y procedimientos con paso de parámetros.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Qué es un algoritmo, sus partes y características
- Tipos de datos y declaración de variables en pseudocódigo
- Uso de operadores aritméticos y relacionales
- Aplicación de estructuras condicionales simples, anidadas y múltiples

» Estructuras de Control Repetitivas

< > Estructuras Repetitivas

¿Para qué sirven?

Las estructuras repetitivas o bucles son construcciones de programación que **permiten ejecutar un bloque de código varias veces** de forma controlada.

Permiten repetir un proceso sin tener que escribir el código muchas veces. Simplifican la escritura de código y se pueden controlar para que se repitan hasta cumplir cierta condición (**mientras**), un número fijo de veces (**para**) o de forma indefinida (**repetir**).

Resultan muy útiles para operaciones reiterativas como procesar listas de elementos, lecturas desde archivos, interacciones con el usuario, etc. Mediante los bucles se puede recorrer arrays, strings, objetos, estructuras de datos complejas, etc.

Hay que tener cuidado de no crear bucles infinitos y controlar adecuadamente la condición de salida.



➤ Estructura Mientras

< > Bucle Mientras

¿Para qué sirve?:

El bucle **Mientras** (while) permite **repetir la ejecución de un bloque de código mientras se cumpla una condición**. La condición se evalúa **antes de ingresar al bucle**. El funcionamiento es el siguiente:

- Primero se evalúa la condición dentro del paréntesis.
- Si la condición se cumple (es **verdadera**), se ejecuta el código definido.
- Una vez termina la ejecución del bloque de código, se vuelve a evaluar la condición.
- Si sigue siendo verdadera, se repite la ejecución del código nuevamente.
- Este proceso se repite iterativamente mientras la condición se cumpla.
- Cuando la condición llega a **false** , **se sale del bucle** y continúa la ejecución normal después de éste.

Es importante que la condición en algún momento se haga falsa para evitar un bucle infinito.



< > Bucle Mientras

Mientras:

Se ejecuta siempre que una **condición sea verdadera** .
Imagina un reloj despertador que suena hasta que presionas el botón de apagado. Mientras la condición de que el reloj siga sonando sea verdadera, el bucle se repetirá.

Mientras que(**condición**)

acción 1

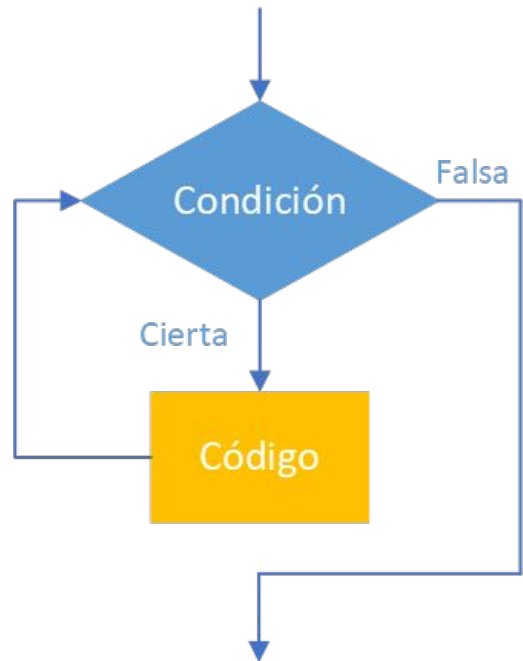
acción 2

acción 3

... acción n

Fin Mientras

Bucle condicional



LIVE CODING

Ejemplo en vivo

Vamos a escribir un algoritmo en el cual se ingrese un **valor límite positivo**, y a continuación solicite números al usuario **hasta que la suma de los números introducidos supere el límite inicial**.

1. Variables: límite, num, suma.
2. Pedir el límite, y luego colocarlo como condición del bucle Mientras
3. Sumar cada número ingresado (reemplazando el valor de “num”) hasta que supere el valor ingresado como límite.

Tiempo: 20 minutos

➤ Estructura “Para”

< > Bucle Para

¿Para qué sirve?:

El bucle **Para** (for) es una estructura de control en programación que permite repetir un bloque de código **un número determinado de veces** . Sus características principales son:

- La **inicialización** define y asigna valor a la **variable de control** del bucle.
Por ejemplo: `int i = 0;`
- La **condición evaluada** en cada iteración determina si el bucle continúa o se detiene. Por ejemplo: `i < 10;`
- El **incremento** modifica el valor de la variable de control en cada iteración.
Por ejemplo: `i++`
- Es muy útil cuando se conoce previamente la cantidad de iteraciones que se requieren.

Es más apropiado que un Mientras, cuando se requiere un número fijo y conocido de repeticiones.



< > Bucle Para

Para:

Es como contar las vueltas que das en una pista de carreras. Especificamos un **valor inicial**, una **condición de finalización** y un **incremento** , y el bucle se ejecutará hasta que se cumpla la condición.

Para (**valor inicial**; **condición de finalización**; **incremento**)

acción 1

acción 2

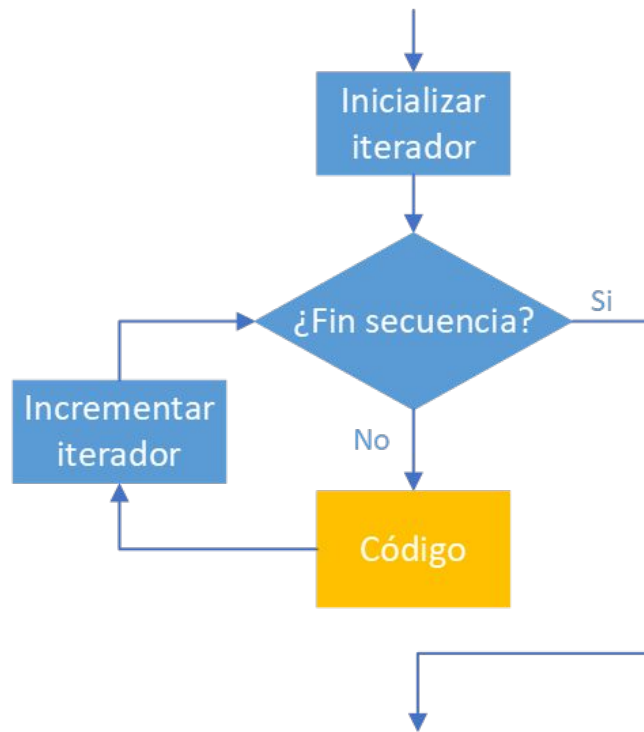
acción 3

....

acción n

Fin_para

Bucle iterativo



< > Bucles

Estas estructuras de control repetitivas nos permiten ahorrar tiempo y evitar repetir líneas de código innecesarias. Son herramientas valiosas para procesar listas de datos, realizar cálculos repetitivos y realizar tareas iterativas de manera eficiente.

Aquí algunas recomendaciones generales sobre **cuándo utilizar cada tipo de bucle** en programación:

Mientras : Cuando no se conoce previamente cuántas iteraciones se necesitan y depende de una condición. Por ejemplo, mientras no se ingrese un valor válido.

Repetir : Cuando se requiere que el bloque de código se ejecute al menos una vez antes de evaluar la condición. Por ejemplo, en un menú de opciones.

Para : Cuando se conoce de antemano la cantidad de iteraciones. Por ejemplo, para iterar sobre todos los elementos de un arreglo. También cuando se necesita llevar un control específico de la variable iteradora. Por ejemplo para imprimir números del 1 al 100.



Imprimiendo tablas:

Vamos a **imprimir la tabla del 2** utilizando un bucle de iteración **Para**.

1. Declaramos la variable contador *i* que va desde 1 hasta 10. Esta será la iteradora del bucle.
2. En cada iteración se calcula el resultado como 2 multiplicado por el valor actual de *i*.
3. Se imprime en pantalla la operación completa con el número 2, el valor de *i* y el resultado.

LIVE CODING

Ejemplo en vivo

Imprimiendo tablas:

Vamos a **imprimir la tabla del 2** utilizando un bucle de iteración **Para**.

4. Al ir incrementando i en cada vuelta, se va calculando y mostrando la tabla de multiplicar del 2.
5. El bucle se repetirá 10 veces, una por cada valor de 1 a 10 en i .

Tiempo: 15 minutos

➤ Diagramas de flujo

< > Diagrama de flujo

¿Qué es y para qué sirve?:

Un diagrama de flujo es una representación gráfica de un algoritmo. Se utiliza para **representar visualmente los pasos o etapas con las que cuenta un algoritmo**, así como el orden o secuencia de las acciones. Los diagramas de flujo sirven para:

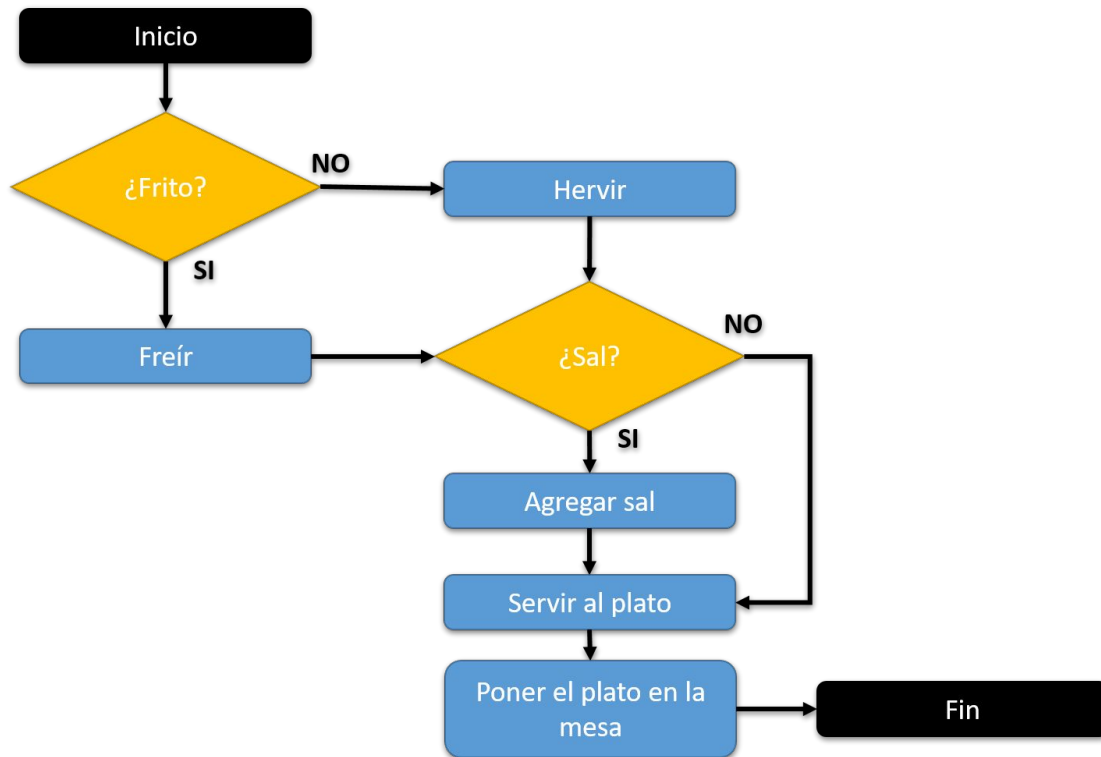
- Describir y documentar procesos de negocio, procedimientos operativos o flujos de trabajo.
- Analizar un proceso actual e identificar oportunidades de mejora.
- Diseñar un proceso nuevo de manera eficiente.
- Comunicar de manera clara y visual un proceso a otras personas.
- Estandarizar la forma en que se realizan ciertas operaciones o tareas.
- Entrenar a nuevos empleados en cómo realizar un proceso.
- Depurar y optimizar un algoritmo antes de codificarlo.



< > Diagramas de Flujo

¿Cómo cocinar un huevo?

Aquí podemos ver un ejemplo de la vida cotidiana representado en un diagrama de flujo.



< > Diagramas de Flujo

Simbología

Además de los símbolos gráficos, se suelen usar palabras clave como **"Inicio"**, **"Fin"**, **"Sí"**, **"No"**, para hacer más clara la interpretación del flujo.

Símbolo	Utilidad o significado
	Línea de flujo o flecha, usado para mostrar la continuidad y orden del proceso.
	Usado para indicar el inicio o fin del proceso o subproceso.
	Indica una actividad del procesos en el diagrama.
	Usado para indicar un decisión de dos opciones (si, no o verdadero, falso).
	Entrada y salida de datos al proceso.
	Documentos del proceso.
	Usado para representar un enlace dentro de otra parte del proceso, este lleva una letra o numero en el centro que es con que empieza el enlace.
	Usado para representar demora en la secuencia de una actividad.
	Usado para representar dos partes del diagrama en diferentes paginas.



Ejercicio N° 1

< > Creando un diagrama

Contexto: 🙌

Vamos a aplicar lo aprendido sobre un ejemplo conocido. Los diagramas de flujo se crean para representar algoritmos. En este caso, el algoritmo será otorgado y sólo deberás centrarte en crear el correspondiente diagrama de flujo teniendo en cuenta la simbología vista anteriormente.

Consigna: 📝

Desarrollar un diagrama de flujo que permita leer dos valores distintos, determinar cuál de los dos valores es el **mayor** y mostrarlo por pantalla. En el siguiente slide podrás ver el algoritmo base.

Tiempo 🕒: 15 minutos



< > Creando un diagrama

Algoritmo:

Inicio

 Inicializar variables: $A = 0$, $B = 0$

 Solicitar la introducción de dos valores distintos

 Leer los dos valores

 Asignarlos a las variables A y B

 Si $A = B$ Entonces vuelve a leer los valores porque deben ser distintos

 Si $A > B$ Entonces

 Escribir A, "Es el mayor"

 De lo contrario: Escribir B, "Es el mayor"

 Fin_Si

Fin



➤ Arreglos

< > Arreglos

¿Qué son y para qué sirven?:

Los arreglos, también llamados **arrays o vectores**, son estructuras de datos que **permiten almacenar múltiples valores** del mismo tipo **en una sola variable**.

Características principales:

- Los valores se almacenan en posiciones o **índices** numerados de forma secuencial (el primer valor se guarda en la posición 0).
- El número de valores que puede contener el arreglo **se define al declarar la variable**.
- Todos los elementos almacenados deben ser del **mismo tipo de datos** (enteros, cadenas, booleanos, etc).
- Se puede acceder de forma directa a cualquier elemento especificando su índice dentro de corchetes.

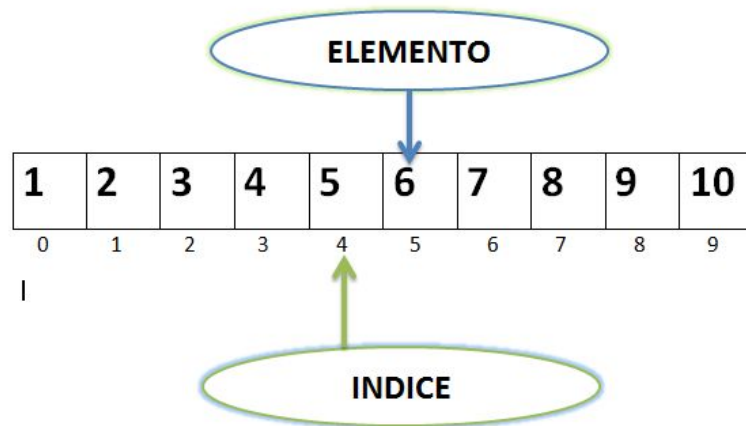


< > Arreglos

Ventajas:

Los arreglos son estructuras de datos clave en programación que permiten manejar múltiples valores como una colección organizada. Son muy útiles para almacenar y procesar datos de forma eficiente.

- Permite almacenar y organizar múltiples datos en una sola estructura.
- Da un acceso rápido y directo a cualquier elemento especificando su índice.
- El código se simplifica **al poder iterar el arreglo con bucles** en lugar de variables individuales.



Arreglos

UNIDIMENSIONALES: Vectores

- Tiene una sola dimensión, es decir, **un solo índice** .
- Los elementos se ordenan en una lista simple.
- Se accede a los elementos con un solo índice.

Ejemplo: `numeros[4] = [1, 2, 3, 4, 5]`

BIDIMENSIONALES: Matrices

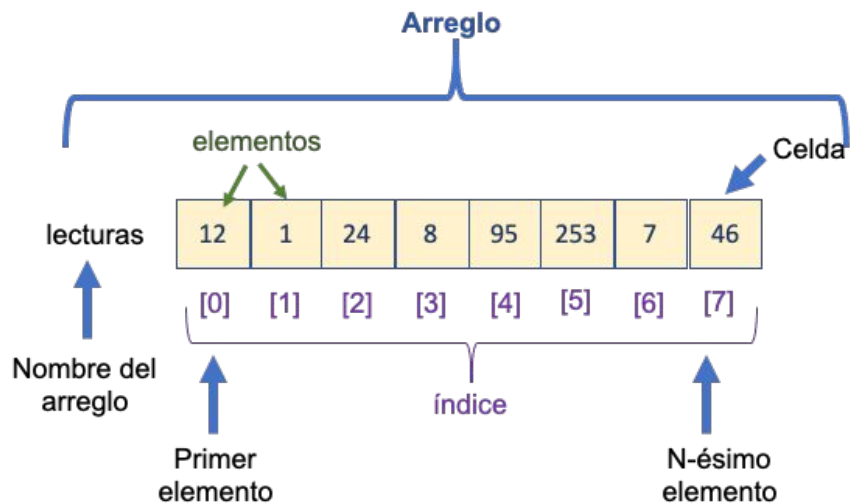
- Tiene dos dimensiones, fila y columna, es decir, **dos índices**.
- Los elementos se ordenan en forma de **tabla** o **matriz** .
- Se accede a los elementos con dos índices, **uno para la fila y otro para la columna** .

Ejemplo: `numeros[2,3] = [[1, 2, 3], [4, 5, 6]]`

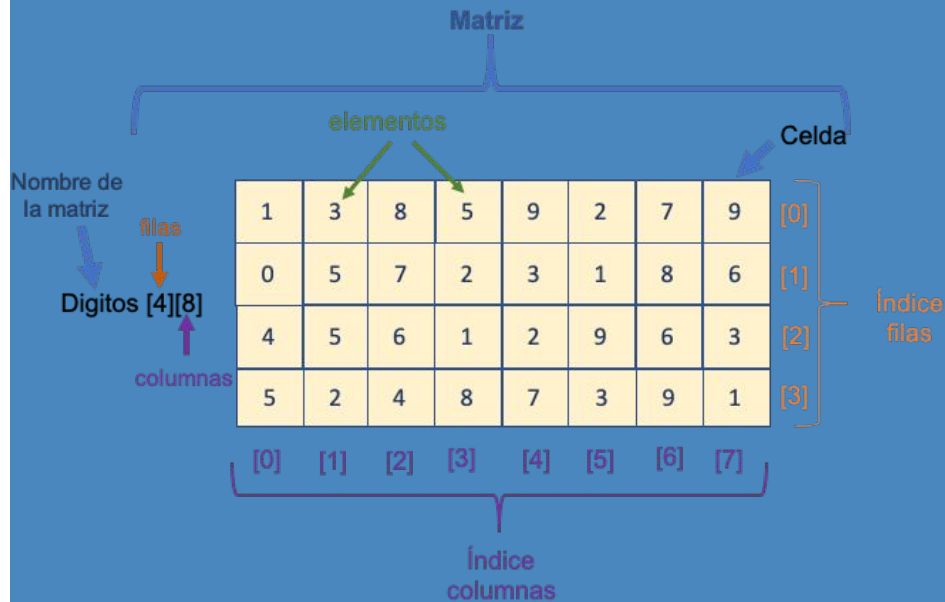
- Para acceder al elemento 5 se usaría `numeros[1,2]`

Arreglos

UNIDIMENSIONALES: Vector



BIDIMENSIONALES: Matriz



< > Arreglos Unidimensionales

Declaración: Ejemplo en pseudocódigo

Para declarar un arreglo, se coloca la palabra reservada Dimensión (para aclarar que la variable es del tipo arreglo), luego el nombre de la variable y a continuación el tamaño del arreglo entre corchetes. En este caso, el arreglo será de tipo entero y guardará sólo datos de este tipo.

Definir numeros como entero

Dimension numeros[5]

Para asignarle un valor a cada índice, se puede hacer de manera manual:

numeros[0] = 5

numeros[1] = 10

numeros[2] = 2

numeros[3] = 17

numeros[4] = 8

O de manera más automatizada, utilizando un bucle Para.



< > Arreglos Unidimensionales

Llenar arreglo de 5 elementos con bucle Para

Usar bucles como for para recorrer arreglos es muy útil para inicializarlos, hacer operaciones y más. Los bucles permiten reducir código repetitivo accediendo secuencialmente a los elementos.

```
Para i = 0 hasta 4
    numeros[i] = i * 10
fin para
```

Esto recorrerá el arreglo asignando a cada posición el valor de su índice multiplicado por 10.

- En la primera iteración $i = 0$, se asigna $0 * 10 = 0$
- En la segunda $i = 1$, se asigna $1 * 10 = 10$
- En la tercera $i = 2$, se asigna $2 * 10 = 20$ Y así sucesivamente hasta completar el arreglo.
- Al final el arreglo numeros tendrá los valores: **[0, 10, 20, 30, 40]**



< > Arreglos Bidimensionales

Declaración : Ejemplo en pseudocódigo

Para declarar una matriz, se coloca la palabra reservada **Dimensión** (para aclarar que la variable es del tipo arreglo), luego el nombre de la variable y a continuación el tamaño del arreglo entre corchetes. La diferencia principal radica en que la matriz tendrá dos índices.

Definir números como entero

Dimension numeros[2,3] (siendo 2 la referencia de las **filas**, y 3 la referencia de las **columnas**)

Para **asignarle un valor a cada índice** , se recomienda utilizar siempre **bucles Para anidados**.

Se utiliza **un bucle Para por cada dimensión que tenga el arreglo**. En este caso tienen dos dimensiones, por lo que se utilizan **dos bucles Para anidados para recorrer tanto filas como columnas** .



< > Arreglos Bidimensionales

Llenar arreglo bidimensional con bucles Para anidados.

```
Para i = 0 hasta 1
  Para j = 0 hasta 2
    numeros[i,j] = i + j
  Fin Para
Fin Para
```

Con este bucle for anidado se recorren ambas dimensiones del arreglo:

- La variable **i** recorre las filas (de 0 a 1)
- La variable **j** recorre las columnas (de 0 a 2)
- En cada iteración se asigna a cada elemento la suma de sus índices i y j.

Al final el arreglo numeros quedará relleno así: $\begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix}$





LIVE CODING

Ejemplo en vivo

Creando arreglos:

- Vamos a realizar un algoritmo que rellene un vector con los 100 primeros números enteros y los muestre por pantalla en orden descendente.
- Luego, vamos a realizar el mismo ejercicio pero rellenando una matriz.

Tiempo: 25 minutos

» Funciones

< > Funciones

¿Qué es una función?

Una función en programación es un **bloque de código reutilizable** que realiza una tarea específica. Funciona como una pequeña unidad de trabajo que acepta entradas (llamadas parámetros) y puede producir salidas (un valor de retorno). Las funciones permiten dividir un programa en partes más pequeñas y manejables, lo que facilita la comprensión, el mantenimiento y la reutilización del código.

Una función es un subprograma que tiene 3 componentes importantes:

- Los **parámetros** son los valores que recibe la función como entrada;
- El **bloque de código** son las operaciones que hace la función;
- El resultado (o **valor de retorno**) es el valor final que entrega la función.



< > Funciones

¿Cómo se declara una función?:

Al definir una función, establecemos su firma, que incluye el **tipo de datos que retorna** la función y los **tipos y nombres de los parámetros** que acepta. Los parámetros son variables locales que **reciben valores cuando se llama a la función**, y estos valores se utilizan dentro del bloque de código para realizar operaciones.

```
nombre_funcion (argumento1 , argumento2)
```

```
sumar (num1, num2)
```

Es posible tener funciones con 0 o más argumentos y pueden estar definidas por el propio usuario o, en muchas ocasiones, ser funciones definidas por el propio compilador.



< > Funciones

¿Cómo se invoca una función?:

Una función definida por el usuario se invoca haciendo referencia a su nombre. En pseudocódigo:

```
...  
nombreFuncion (lista de parámetros reales o actuales)  
...
```

La sentencia **nombreFuncion** acompañada de los **parámetros** es la que inicia la ejecución de la función. El control de ejecución lo toma la función, ejecuta secuencialmente cada una de sus sentencias, y cuando termina de ejecutarse, le devuelve el control al programa llamador



LIVE CODING

Ejemplo en vivo

Creando una función en pseudocódigo:

Vamos a crear un algoritmo que a través de una función convierta una cantidad de euros introducida por teclado a otra moneda, estas pueden ser a dólares, yenes o libras. La función tendrá como parámetros, la cantidad de euros y la moneda a convertir (cadena).

El cambio de divisas es:

- * 0.86 libras es un 1 €
- * 1.28611 \$ es un 1 €
- * 129.852 yenes es un 1 €



LIVE CODING

Ejemplo en vivo

Creando una función: Paso a paso

1. *Primero debes crear el algoritmo y pedirle al usuario el monto en euros*
2. *Luego llamar a la función.*
3. *La función debe codificarse teniendo en cuenta que su parámetro será (euro)*
4. *Devolver el resultado de todas las conversiones.*

Tiempo: 20 minutos



Momento:

Time-out!



5 -10 min.





Ejercicio N° 2

< > Calculadora 3.0

Contexto: 🙌

Vamos a mejorar la calculadora que hemos estado creando en diferentes ejercicios. En este caso, vamos a reutilizar código para que el cuerpo del algoritmo quede más simple. Para esto, haremos uso de funciones.

Consigna: 🖋️

Crea un algoritmo que le pida dos números al usuario y este pueda elegir entre sumar, restar, multiplicar y dividir. La aplicación debe tener una función para cada operación matemática y deben devolver sus resultados.

Tiempo 🕒: 20 minutos

Paso a paso: ⚙️

- Definir una función para sumar que reciba dos números como parámetros y retorne la suma de ambos. Realizar lo mismo con el resto de las operaciones.
- Pedir al usuario que ingrese dos números y almacenarlos en variables num1 y num2
- Mostrar un menú con las opciones: sumar, restar, multiplicar y dividir
- Leer la opción elegida por el usuario y almacenarla en una variable opción
- Utilizar un condicional Según para ejecutar la función según la opción elegida.
- En cada caso del Según, llamar a la función correspondiente para imprimir su valor.



➤ Procedimientos

< > Procedimientos

¿Qué es un procedimiento?:

Un procedimiento es un bloque de código que realiza una tarea específica, **pero no retorna ningún valor** . Es por esto que los procedimientos no especifican tipo de dato de retorno. También deben ser declarados obligatoriamente antes de que puedan ser llamados en el cuerpo del programa principal.

Todo procedimiento, al igual que un programa principal, consta de una cabecera, que proporciona su nombre y sus parámetros de comunicación; de una sección de declaraciones locales y el cuerpo de sentencias ejecutables.



< > Procedimientos

Las principales diferencias entre procedimientos y funciones son:

- Un procedimiento es un bloque de código que realiza una tarea específica, pero no retorna ningún valor. Una función también realiza una tarea, pero debe retornar un valor.
- Las funciones deben declararse indicando el tipo de dato del return, en cambio, los procedimientos no especifican tipo de retorno.
- Las funciones se pueden llamar desde dentro de expresiones, por ejemplo, para pasarlas como parámetro a otra función. Los procedimientos no se pueden usar de esta forma.

En general, si una tarea requiere devolver un valor, es mejor usar una función. Si solo se necesita ejecutar un bloque de código, un procedimiento es suficiente.



< > Procedimientos

¿Cómo se declara un procedimiento?

En pseudocódigo, la declaración de un procedimiento tiene la siguiente estructura:

```
PROCEDIMIENTO nombreProcedimiento(parámetros)
```

```
    Instrucciones
```

```
FINPROCEDIMIENTO
```

- **nombreProcedimiento** : es el identificador único con el que se llamará al procedimiento.
- **parámetros** (opcional): lista de parámetros o variables entre paréntesis que recibe el procedimiento.
- **Instrucciones** : son las acciones que realiza el procedimiento, pueden ser asignaciones, operaciones, lectura de datos, etc.



➤ Paso de parámetros

< > **Parámetros**

Paso de parámetros a subprogramas

Los métodos más empleados para realizar el paso de parámetros son:

- o **Paso por valor** (parámetro valor)
- o **Paso por referencia o dirección** (parámetro variable)

Los parámetros formales (locales al subprograma) reciben como valores iniciales los valores de los parámetros reales, y con ellos se ejecutan las acciones descritas en el subprograma.



Parámetros

Paso por valor:

Son los parámetros **unidireccionales** , que se usan para dar información al subprograma, pero no pueden devolver valores desde el mismo. **Aunque el procedimiento les cambie su valor, este nuevo valor no se refleja en el programa** llamador. Esto se debe a que en la llamada al subprograma, se le asigna el valor del parámetro real a la variable que representa al parámetro formal correspondiente, dicho de otra forma, **se crea una copia del parámetro real**.

Paso por referencia:

Son los parámetros que **sólo reciben valor en el subprograma** , o bien proporcionan valor al subprograma y reciben un valor nuevo en el mismo. Así, **todo cambio realizado sobre los parámetros formales se refleja en los parámetros reales correspondientes** . Se los considera como parámetros **bidireccionales** o variables, ya que son de Entrada y/o Salida.

Parámetros

En funciones:

Una función puede tener parámetros variables además de parámetros valor en la lista de parámetros formales. Una función puede cambiar el contenido de una variable global y ejecutar instrucciones de entrada/salida. Estas operaciones se conocen como parámetros laterales, y se deben evitar.

En procedimientos

Si un procedimiento modifica una variable global (distinta de un parámetro real), éste es un efecto lateral. Por ello, excepto en contadas ocasiones, no debe aparecer en la declaración del procedimiento. Si se necesita una variable temporal en un procedimiento, se debe utilizar una variable local, no una global. Si se desea que el procedimiento modifique el valor de una variable global, utilízala como el parámetro real en una llamada al procedimiento.

LIVE CODING

Ejemplo en vivo

Creando un procedimiento en pseudocódigo:

Vamos a crear un procedimiento que permita ingresar los días de la semana por teclado y guardarlos en un arreglo. Para finalizar, imprimimos el arreglo por pantalla.

Tiempo: 20 minutos

LIVE CODING

Ejemplo en vivo

1. Definir el arreglo "dias[7]"
2. Definir el procedimiento y el parámetro: "cargarDias(dias)"
3. Dentro del procedimiento, iterar para pedir los días:
4. Dentro del ciclo, leer un día con "LEER dias[i]"
5. Al finalizar la carga, recorrer el arreglo e imprimirlo:
6. Usar otro ciclo para escribir cada día con "ESCRIBIR dias[i]"
7. Invocar el procedimiento desde el programa principal.

Tiempo: 20 minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Utilizamos estructuras Mientras y Para para resolver problemas iterativos**
- ✓ **Representamos algoritmos con diagramas de flujo**
- ✓ **Creamos y manipulamos arreglos simples y matrices con pseudocódigo**
- ✓ **Implementamos funciones que devuelven resultados**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

